# Experiment 6

**Student Name: Deepanshu Mandhyan** **UID: 23BCS12327**
**Branch: CSE** **Section/Group: KRG 3-A**
**Semester: 6th** **Date of Performance:27/02/2026**
**Subject Name: System Design** **Subject Code: 23CSH-314**

## 1. Aim:

To develop a highly scalable video streaming platform, similar to YouTube, capable of handling large volumes of video uploads, providing adaptive bitrate streaming, and enabling real-time social interactions while ensuring high availability and minimal latency.

## 2. Objective:

- Understand the architecture of a large-scale media streaming system.
- Determine the functional requirements, such as direct cloud-based uploads and personalized content feeds.
- Identify non-functional requirements, including CDN caching, high throughput, and strict latency considerations.
- Examine how synchronous API requests can be decoupled using event-driven stream processing.
- Design RESTful APIs for managing videos, enabling search functionality, and supporting user interactions.

## 3. Tools Used:

- **Draw.io –** Used for designing the system's high-level architecture (HLD).
- **Apache Kafka –** Used for event streaming and message processing**.**
- **CDC Pipeline –** Used to synchronize product data with Elasticsearch**.**

## 4. System Requirements:

## A. Functional Requirements

- The system should enable users to sign up, log in, and manage their accounts.
- The system should support efficient uploading of large raw video files.
- The system should provide smooth video playback using Adaptive Bitrate Streaming based on the user's network conditions.
- The system should facilitate real-time social interactions such as likes, dislikes, and comments.
- The system should allow users to search for videos using titles or metadata.
- The system should create a personalized homepage feed based on the user's watch history.

## B. Non – Functional Requirements

1. The system must be capable of managing extremely high throughput, supporting concurrent video uploads and millions of simultaneous viewers.
2. High availability should be ensured by eliminating any single point of failure within the system.
3. The system should implement an eventual consistency model for social metrics such as view counts and likes to reduce database write pressure.
4. Video delivery latency should be minimized through the use of edge caching and a globally distributed CDN.
5. The transcoding pipeline should be horizontally scalable to efficiently handle sudden spikes in video uploads.

## 5. API Diagram:

**-API Design**

**1. <u>Authentication API</u>**

A. User Registration: POST /api/v1/auth/register
B. User Login: POST /api/v1/auth/login

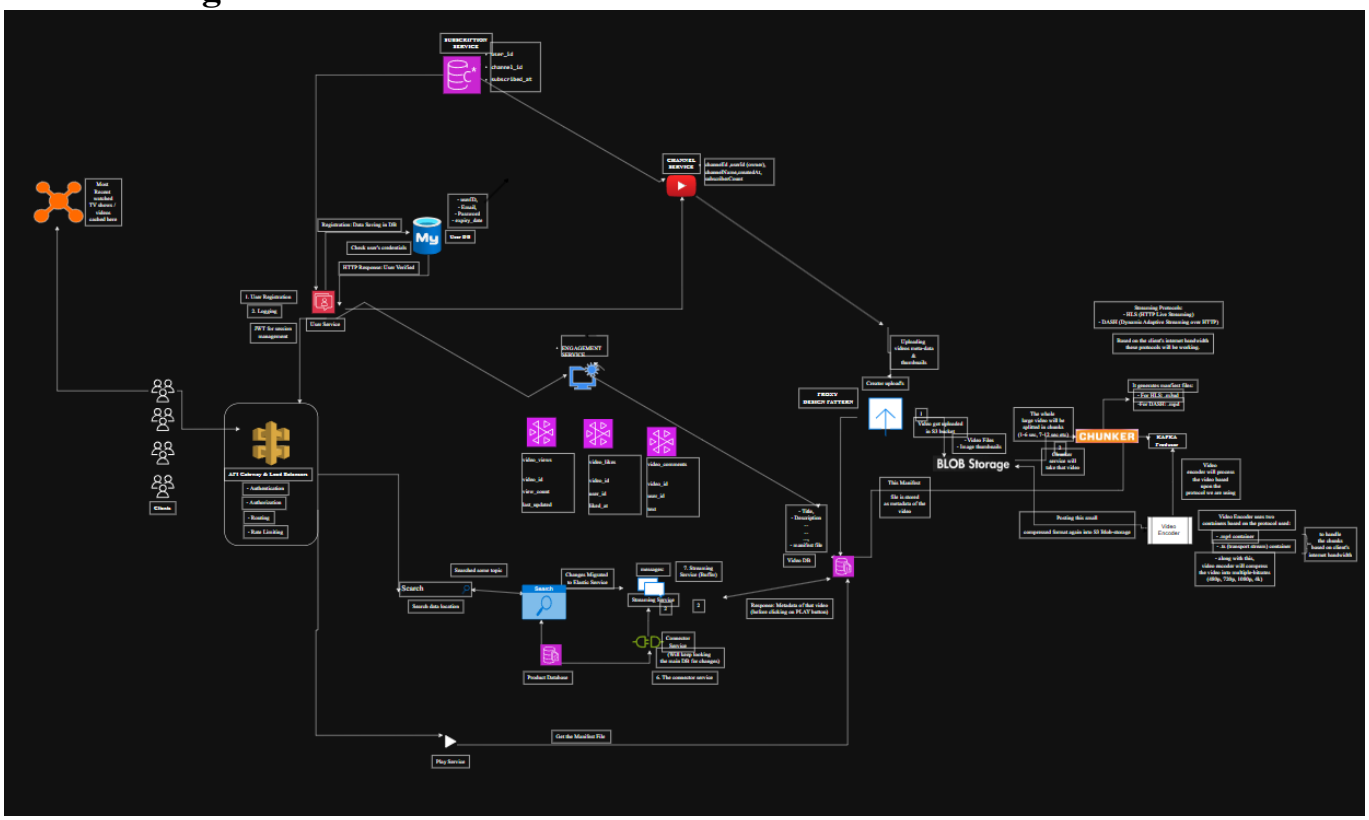
**2. <u>Video and Feed API</u>**

A. Start Video Upload: POST /api/v1/videos/upload/init (Returns an S3 pre-signed URL)
B. Retrieve Video Metadata: GET /api/v1/videos/{video_id}
C. Fetch Personalized Feed: GET /api/v1/feed (Supports pagination)
D. Search Videos: GET /api/v1/search?q={query}

**3. <u>Interaction API</u>**

A. Like a Video: POST /api/v1/videos/{video_id}/like
B. Post a Comment: POST /api/v1/videos/{video_id}/comments
C. Retrieve Comments: GET /api/v1/videos/{video_id}/comment

## 6. HLD Diagram:




## 7. Learning Outcomes:

- Developed a highly scalable, event-driven architecture for a video streaming platform.
- Determined the key functional and non-functional requirements necessary for efficient media delivery.
- Designed RESTful APIs and implemented decoupled asynchronous processing pipelines.
- Gained an understanding of how CDNs, adaptive bitrate streaming, and stream processing are used to support large-scale systems.